

UNIVERSIDAD NACIONAL DEL NORDESTE

Facultad de Ciencias Exactas y Naturales y de Agrimensura

Informe Técnico Analítico

Sistema de Semáforos Inteligentes

Licenciatura en sistemas de Información.
Paradigmas y Lenguajes de Programación.

Profesores:

- Msc. Ricardo, Monzón
- Ringa, Mónica
- Rodríguez, Leandro
- Mascazzini, Matías
- Ramirez, Walter

Integrantes del Grupo 40:

- Fernandez Casado, Guido Sebastian
- Romero, Enzo
- Scher, Emily Giuliana
- Soto, Nahuel Federico
- Vera, Gaston

Fundamentos del diseño funcional utilizado

Dentro del paradigma funciona, se desarrolla un marco de pensamiento declarativo que conforma el cómputo como la evaluación de funciones matemáticas, evitando de manera explícita el cambio de estado y la mutabilidad de los datos.

Para el desarrollo de este sistema de semáforos, se aplicaron las siguientes reglas:

- **Inmutabilidad Absoluta y Flujo de Estados:** En las funciones desarrolladas, se prohíbe el uso de variables globales mutables y operadores destructivos de asignación.
La información tiene que ser tratada como datos de lectura y las transiciones deben producir nuevos valores sin destruir los anteriores.
- **Erradicación de Bucles Imperativos:** Se descarta el uso de estructuras de control iterativas como loop, do-while, do-while o while. Toda iteración, procesamiento de flujos o cálculo acumulativo se tiene que resolver mediante mecanismos funcionales.
- **Autodocumentación y Clasificación:** Cada función implementada, tanto en Lisp como en el lenguaje asignado (Scala), tiene que contar con un encabezado

taxonómico obligatorio en sus comentarios. Este bloque define tres atributos esenciales de las funciones:

Naturaleza: Clasificación entre función Pura (ausencia total de efectos colaterales y sujeta a determinismo matemático) o Impura (aquellas dedicadas al sistema de auditoría y *logging* en archivos o terminal).

Estrategia de Control: Identificación del modelo de ejecución, tipificando si es Recursiva Simple, Recursiva de Cola, si delega en Funciones de Orden Superior o si actúa como Función Predicado.

Impacto en Memoria: Evaluación de la persistencia de las estructuras de datos, determinando si el comportamiento es Destructivo (mutación de celdas existentes) o No Destructivo (generación de nuevas estructuras preservando la inmutabilidad).

Justificación de librería seleccionada: local-time

Para esta etapa, decidimos utilizar la librería externa local-time, del gestor de paquetes Quicklisp, se detallan los fundamentos técnicos:

- **Abstracción del Formato Unix (Epoch):** El sistema base procesa temporalidades mediante estampas de tiempo Unix (enteros que contabilizan segundos transcurridos desde 1970). Si bien este formato es óptimo para el cómputo interno y el determinismo de la función *timer*, resulta hermético e ineficiente para la interpretación humana en el Sistema de Auditoría.
- **Optimización del Análisis Forense:** La inclusión de *local-time* permite interceptar dicho entero y transformarlo dinámicamente en un objeto de tiempo estructurado, facilitando su conversión a una cadena de caracteres legible (v.g., [2026-05-16 14:30:00]). Esto optimiza drásticamente la utilidad del mecanismo de *logging*, permitiendo a un analista forense reconstruir la cronología de transiciones de manera inmediata sin depender de conversores externos.
- **Compatibilidad con Extensiones de Persistencia:** La estandarización del formato de fecha provista por la librería simplifica la implementación de la extensión de persistencia de datos (Iteración 2), asegurando que las entradas escritas en el archivo de texto plano *informe-ejecucion-semaforo.txt* sigan una estructura uniforme, profesional y normalizada internacionalmente.

Bitácora de Depuración

Durante el desarrollo de las fases y requerimientos del Sistema, el equipo documentó y resolvió los siguientes errores técnicos:

- **Bug N°1:** Error de Evaluación por Confusión de Símbolo como Función
 - **Problema:** Al ejecutar la función “transicion” para retornar el par ordenado de estado y acción, el entorno de ejecución arrojaba un error crítico indicando que la variable local *c1* no correspondía a una función definida.

- **Causa:** En la expresión (list (c1 (format nil ...))), el intérprete de Common Lisp sigue la regla de evaluación de prefijos (Lisp-1/Lisp-2). Al encontrar un paréntesis izquierdo antes del símbolo c1, intentó evaluar a c1 como el identificador de una función en lugar de un argumento simbólico, interrumpiendo el flujo del programa.
- **Resolución:** Se suprimió la invocación errónea removiendo los paréntesis internos redundantes. La estructura se modificó hacia (list color-actual (format nil ...)), permitiendo que list agrupe correctamente los dos átomos independientes sin forzar una evaluación funcional inexistente.

Problema: (defun transicion (c1 c2) (let ((r 'en-rojo) (y 'en-amarillo) (v 'en-verde))
 (cond
 ((eq c1 r) (list (c1 (format nil "cambiar a ~A" c2))))
 ((eq c1 y) (list (c1 (format nil "cambiar a ~A" c2))))
 ((eq c1 v) (list (c1 (format nil "cambiar a ~A" c2))))
)); al poner (c1 lisp interpreta como que c1 es una funcion, y al no serlo, se rompe.

- **Bug N°2:** Error de Nomenclatura al Acoplar timer y transición
 - **Problema:** Al conectar la lógica del temporizador con la función transicion, el programa fallaba con un error de variable no vinculada (unbound variable) al intentar usar los símbolos c1 y c2.
 - **Causa:** Se cambiaron los nombres de los parámetros en la firma de la función transicion para usar nombres más descriptivos (color-actual y cambiar-a). Sin embargo, dentro del condicional cond se olvidó actualizar el código y se siguieron llamando las variables viejas c1 y c2, las cuales ya no existían en ese ámbito.
 - **Resolución:** Se corrigió el cuerpo de las condiciones reemplazando c1 y c2 por las variables correctas pasadas por parámetro: color-actual y cambiar-a. Esto unificó los nombres en toda la estructura y permitió que los datos fluyeran de forma limpia y sin errores.

Problema: (cond
 ((and (eq color-actual r) (eq cambiar-a 'verde)) (list c1 (format nil "cambiar a ~A" c2)))
 ((and (eq color-actual y) (eq cambiar-a 'rojo)) (list c1 (format nil "cambiar a ~A" c2)))
 ((and (eq color-actual g) (eq cambiar-a 'amarillo)) (list c1 (format nil "cambiar a ~A" c2)))
 (t "Estado invalido"))))

- **Bug N°3:** Clausura Sintáctica Errónea en Estructuras Condicionales (cond)
 - **Problema:** Durante la fase de pruebas del Requerimiento 3 (Sistema de Auditoría), el intérprete de Common Lisp interrumpía la carga del archivo arrojando fallos sintácticos y omitiendo la evaluación de los estados intermedios (fases de color amarillo y la cláusula por defecto t).
 - **Causa:** El equipo incurrió en un error de anidamiento al introducir un paréntesis de cierre redundante al finalizar el cuerpo de la primera condición: (list (format nil ...));. En la sintaxis de la macro cond, este exceso provocó la

clausura prematura de la estructura de control total. En consecuencia, las cláusulas subsecuentes quedaron aisladas del ámbito del cond y fueron interpretadas de manera errónea como S-expresiones sueltas e inválidas dentro del bloque let.

- **Resolución:** Se corrigió el balanceo de paréntesis, reestructurando las cláusulas para que se mantuvieran de manera homogénea dentro de los límites del cond. Posteriormente, aplicando los principios de simplificación y modularidad del paradigma funcional, se determinó que dado que las funciones subyacentes timer y transicion ya garantizan formalmente la integridad matemática y la validez de las combinaciones de estado, el árbol de decisiones duplicado en el módulo de logging era redundante. Se procedió a reescribir la función eliminando el cond redundante y abstrayendo la salida directa de las variables enlazadas.

- **Bug N°4:** Retorno Múltiple Inesperado en Operaciones de Planificación Temporal

- **Problema:** Al probar el Requerimiento 5 (ciclos-por-tiempo), el sistema devuelve dos valores numéricos independientes (por ejemplo, el cociente entero 3 y la fracción flotante restante 0.6111), lo que generaba dudas sobre el impacto en el determinismo funcional.
- **Causa:** La función primitiva floor en Common Lisp está diseñada de forma nativa para proporcionar múltiples valores de retorno utilizando la pila del sistema (el cociente y el residuo).
- **Resolución:** Tras un análisis metalógico, se determinó que la naturaleza de Common Lisp descarta automáticamente los valores secundarios cuando la función se somete a composición funcional directa con otras expresiones receptoras. No obstante, para garantizar una consistencia matemática absoluta y evitar discrepancias en la posterior traducción a otros lenguajes, se documentó que el valor principal (entero) es el único absorbido por el flujo de datos.

- **Bug N°5:** Invocación Sintáctica Errónea y Flujo de Formateo en la Librería local-time

- **Problema:** En el desarrollo de la Fase 2, al intentar inicializar el Sistema de Auditoría adaptado con la stampa de tiempo legible, se generaban interrupciones del sistema al invocar la librería mediante la sintaxis literal (local-time:120129).
- **Causa:** El equipo desconocía la semántica interna y el flujo de procesamiento impuesto por el paquete local-time. En Common Lisp, el operador actúa como calificador de paquetes; al proveer un valor numérico continuo, el motor SBCL intentaba localizar un símbolo o función homónima inexistente dentro del espacio de nombres. Adicionalmente, se asumía erróneamente que la librería realizaba una conversión directa a cadena a partir de un entero.
- **Resolución:** Se reestructuró el flujo de auditoría dividiendo el proceso de obtención y formateo en dos etapas puras e inmutables. En primer lugar, se incorporó la función primitiva local-time:unix-to-timestamp, la cual procesa la stampa temporal Unix (entero) y genera una estructura de datos interna

compatible. En segundo lugar, dicho objeto intermedio se somete a la función de orden superior `local-time:format-timestring`, pasando el argumento `nil` como destino de salida para forzar el retorno directo de una nueva cadena de caracteres (string) formateada con el estándar ISO solicitado ([YYYY-MM-DD HH:MM:SS]).

- **Bug N°6:** Invocación Sintáctica Errónea y Flujo de Formateo en la Librería `local-time`
 - **Problema:** Al evaluar matemáticamente el Requerimiento 6 (Informe de Distribución Temporal), los cálculos porcentuales estáticos arrojaban valores inexactos y desajustados al simular bloques temporales sucesivos superiores a una hora.
 - **Causa:** Un ciclo semafórico completo bajo las reglas de negocio actuales (90s Rojo + 6s Amarillo + 120s Verde) acumula una duración exacta de 216 segundos. Al analizar el intervalo estipulado de 1 hora (3600 segundos), la división aritmética resulta en 16.6666... ciclos ($3600 / 216$), evidenciando que dentro de la primera hora se ejecutan 16 ciclos íntegros y resta un remanente asincrónico de 144 segundos. Este residuo equivale a $\frac{2}{3}$ de una nueva iteración. En consecuencia, la segunda hora no se inicia de manera limpia en el segundo cero del ciclo rojo, sino que arrastra el tercio final truncado (72 segundos) de la hora precedente, alterando la distribución de los porcentajes de cada color de hora en hora.
 - **Resolución:** Tras un modelado analítico, se determinó la existencia de una ventana de neutralización de desfase de 3 horas exactas, momento en el cual el residuo acumulado ($144s \times 3 = 432s$) completa exactamente dos ciclos perfectos de 216 segundos ($2 \times 216 = 432$). Con el fin de evitar un análisis erróneo o "hardcodeado" aplicable únicamente a la primera hora, se rediseñó la función para que compute la distribución porcentual en función de una variable paramétrica *N* (hora específica indexada), aplicando operaciones de módulo aritmético sobre el flujo acumulado para determinar el punto de inicio exacto de las fases y proveer un reporte logístico preciso.

Introducción a Scala y su efecto.

Scala (cuyo nombre proviene de Scalable Language) es un lenguaje de programación de propósito general de tipado estático estricto, diseñado por el científico de la computación Martin Odersky y lanzado a principios de la década de 2000. Su principal fortaleza radica en que fusiona de manera nativa dos paradigmas fundamentales: la Programación Orientada a Objetos (POO) y la Programación Funcional, permitiendo transicionar de un enfoque a otro de forma fluida según las necesidades de la arquitectura.

Debido a su robustez, tipado estricto y gran manejo de la concurrencia, Scala se ha convertido en el estándar de facto para sectores tecnológicos de alta exigencia:

- **Big Data y Procesamiento Distribuido:** Es el lenguaje nativo en el que están escritas herramientas líderes de la industria como *Apache Spark* y *Apache Kafka*, fundamentales para el análisis de datos masivos en tiempo real.

- **Sistemas Backend de Alta Disponibilidad:** Muy utilizado en el sector de las finanzas, el comercio electrónico y el *streaming* para construir arquitecturas capaces de soportar millones de transiciones simultáneas.

Empresas Referentes que Utilizan Scala

- **Netflix:** Lo utiliza para gestionar la lógica de sus microservicios de recomendación de contenido y flujos de datos a gran escala.
- **Twitter (X):** Migró gran parte de su arquitectura backend original de Ruby a Scala para solucionar sus problemas históricos de escalabilidad y caídas del sistema.
- **LinkedIn:** Emplea Scala junto con Kafka para procesar e indexar en tiempo real los millones de eventos y publicaciones que generan sus usuarios globalmente.

1. Scala combina la Programación Orientada a Objetos (POO) con la Funcional. ¿Cómo estructuraron el semáforo? ¿Usaron una clase tradicional, un object (Singleton), o case classes? Justifiquen desde el diseño funcional.

En nuestra implementación estructuramos el semáforo mediante un (object) llamado Semáforo , dentro del cual agrupamos las funciones transición y timer migradas desde common lisp a Scala.

Elegimos usar un objeto porque no necesitamos crear múltiples instancias del semáforo ni representar objetos con identidad propia. es decir no modelamos varios semáforos independientes con atributos modificables, sino un conjunto de operaciones relacionadas con la lógica del sistema, en este caso el object funciona como un módulo contenedor de funciones.

Desde el diseño funcional resulta adecuado porque las funciones implementadas son puras: reciben parámetros de entrada, calculan un resultado y lo devuelve sin modificar variables globales, sin alterar estructuras existentes y sin depender de estado interno mutable. por ejemplo, timer recibe un tiempo y siempre devuelve el mismo color para ese mismo valor; del mismo modo, transición recibe dos estados y siempre devuelve la misma respuesta para esa combinación (tupla).

No usamos class porque no necesitamos crear objetos con estilo propio como semáforos con colores modificables u otro atributo, tampoco usamos case class ya que no vamos a modelar datos estructurados,

Se nos pedía implementar el comportamiento funcional, el uso del object nos permitió aprovechar una característica de la orientación a objetos de Scala para organizar el código , pero manteniendo un enfoque funcional basado en funciones puras , no destructivas y sin estado mutable.

2. Comparen la manipulación de listas en Scala (métodos como .map o .filter) contra las funciones de orden superior de Common Lisp. ¿Cuál resulta más legible y por qué?

En Scala la manipulación de listas se realiza mediante métodos asociados directamente a la colección, como .map y .filter. por ejemplo, una transformación puede escribirse como “ lista.map (x => x * 2)” lo que permite leer la operación de izquierda a derecha: primero aparece la lista luego la acción que se aplica a cada elemento. Esto hace que el código

resulte bastante claro para quienes vienen de lenguajes modernos. Comparado con common lisp, conceptualmente `lista.map (funcion )` cumple con el mismo rol la función de orden superior `(mapcar 'funcion lista)` o `(mapcar (lambda x (funcion sobrex))lista )` que es aplicar una función a cada elemento sin modificar destructivamente la lista original.

Consideramos que para operaciones simples sobre listas, Scala resulta más legible por que la notación es más compacta y cercana al lenguaje natural. Sin embargo en common lisp tiene la ventaja de mantener una estructura más homogénea, donde tanto las funciones comunes como las funciones de orden superior se escriben bajo la misma lógica de listas y expresiones prefijadas.

En conclusión ambas funciones tanto `lista.map(funcion)` como `(mapcar funcion lista)` son equivalentes y favorecen un estilo no destructivo. Para este trabajo nos resultó más legible la sintaxis de Scala en operaciones simples, mientras que Common Lisp se destaca por su uniformidad y coherencia dentro del paradigma funcional.